

Ex.No:27

Experiment : Prolog Program to Implement Best First Search Algorithm

Aim

To write a Prolog program to implement the Best First Search algorithm and find a path from a starting node to a goal node using heuristic values.

Objectives

- To understand the Best First Search algorithm.
- To use heuristic values for searching.
- To find a path from the start node to the goal node.
- To implement the algorithm using Prolog.

Algorithm

1. Start.
2. Define the graph and heuristic values.
3. Set the starting node.
4. Select the node with the smallest heuristic value.
5. Check whether it is the goal node.
6. If it is not the goal, expand its neighboring nodes.
7. Add the neighboring nodes to the search list.
8. Select the node with the lowest heuristic value.
9. Repeat until the goal is reached.
10. Display the path.
11. Stop.

Program

% Best First Search

edge(a, b).

edge(a, c).

edge(b, d).

edge(b, e).

edge(c, f).

edge(c, g).

edge(d, h).

edge(e, h).

edge(f, h).

edge(g, h).

heuristic(a, 7).

heuristic(b, 6).

heuristic(c, 5).

heuristic(d, 4).

heuristic(e, 3).

heuristic(f, 2).

heuristic(g, 1).

heuristic(h, 0).

best_first(Start, Goal, Path) :-

 search([Start], Goal, [], RevPath),

 reverse(RevPath, Path).

search([Goal|_], Goal, Visited, [Goal|Visited]).

search(Open, Goal, Visited, Path) :-

 Open = [Current|Rest],

 findall(H-Next,

 (edge(Current, Next),

 \+ member(Next, Visited),

 heuristic(Next, H)),

```

    Children),
    add_children(Children, Rest, NewOpen),
    search(NewOpen, Goal, [Current|Visited], Path).
add_children([], Open, Open).
add_children([H-Node|Rest], Open, NewOpen) :-
    insert_by_heuristic(H-Node, Open, Temp),
    add_children(Rest, Temp, NewOpen).
insert_by_heuristic(H-Node, [], [Node]) :-
    heuristic(Node, H).
insert_by_heuristic(H-Node, [First|Rest], [Node,First|Rest]) :-
    heuristic(First, H1),
    H =< H1.
insert_by_heuristic(H-Node, [First|Rest], [First|NewRest]) :-
    insert_by_heuristic(H-Node, Rest, NewRest).

```

Sample Input

```
?- best_first(a, h, Path).
```

Sample Output

```

GNU Prolog console
File Edit Terminal Prolog Help
GNU Prolog 1.5.0 (64 bits)
Compiled Jul 8 2021, 12:33:56 with cl
Copyright (C) 1999-2021 Daniel Diaz

| ?- consult('C:/Users/Sireesha/OneDrive/Documents/AI Prolog Experiments/Best First Search Algorithm.pl').
compiling C:/Users/Sireesha/OneDrive/Documents/AI Prolog Experiments/Best First Search Algorithm.pl for byte code
C:/Users/Sireesha/OneDrive/Documents/AI Prolog Experiments/Best First Search Algorithm.pl compiled, 52 lines read
yes
| ?- best_first(a, h, Path).
Path = [a,c,g,h] ?
yes
| ?- |

```

Result

Thus, the Best First Search algorithm was successfully implemented in Prolog, and a path from the starting node a to the goal node h was obtained using heuristic values.

Ex.No:28

Experiment : Prolog Program for Medical Diagnosis

Aim

To write a Prolog program to identify a possible disease based on the symptoms given by the user.

Objectives

- To understand facts and rules in Prolog.
- To represent symptoms and diseases.
- To diagnose a disease based on given symptoms.
- To use Prolog queries for diagnosis.

Algorithm

1. Start.
2. Define symptoms for different diseases.
3. Define rules for diagnosing diseases.
4. Enter the symptoms as a query.
5. Prolog checks the matching rules.
6. Display the possible disease.
7. Stop.

Program

```
% Medical Diagnosis System
```

```
symptom(fever).
```

```
symptom(cough).
```

```
symptom(cold).
```

```
symptom(headache).
```

```
symptom(body_pain).
```

```
symptom(sneezing).
```

symptom(sore_throat).

disease(flu) :-

 symptom(fever),

 symptom(cough),

 symptom(body_pain).

disease(common_cold) :-

 symptom(cold),

 symptom(sneezing),

 symptom(sore_throat).

disease(covid) :-

 symptom(fever),

 symptom(cough),

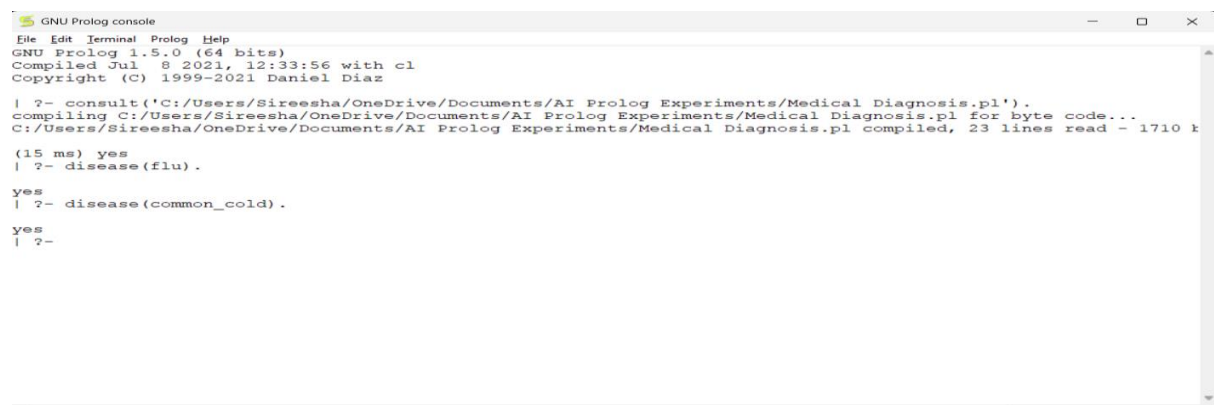
 symptom(sore_throat).

Sample Input

?- disease(flu).

?-disease(common_cold).

Sample Output



```
GNU Prolog console
File Edit Terminal Prolog Help
GNU Prolog 1.5.0 (64 bits)
Compiled Jul  8 2021, 12:33:56 with cl
Copyright (C) 1999-2021 Daniel Diaz

| ?- consult('C:/Users/Sireesha/OneDrive/Documents/AI Prolog Experiments/Medical Diagnosis.pl').
compiling C:/Users/Sireesha/OneDrive/Documents/AI Prolog Experiments/Medical Diagnosis.pl for byte code...
C:/Users/Sireesha/OneDrive/Documents/AI Prolog Experiments/Medical Diagnosis.pl compiled, 23 lines read - 1710 b

(15 ms) yes
| ?- disease(flu) .

yes
| ?- disease(common_cold) .

yes
| ?-
```

Result

Thus, the Prolog program for Medical Diagnosis was successfully implemented using symptoms and rules.

Ex.No:29

Experiment 29: Prolog Program for Forward Chaining

Aim

To write a Prolog program to implement **Forward Chaining** and derive a conclusion from known facts and rules.

Objectives

- To understand Forward Chaining.
- To define facts and rules.
- To derive new facts from existing facts.
- To perform the required queries.

Algorithm

1. Start.
2. Define the initial facts.
3. Define rules based on the facts.
4. Apply the rules to derive new facts.
5. Continue until no new facts can be derived.
6. Query the required conclusion.
7. Display the result.
8. Stop.

Program

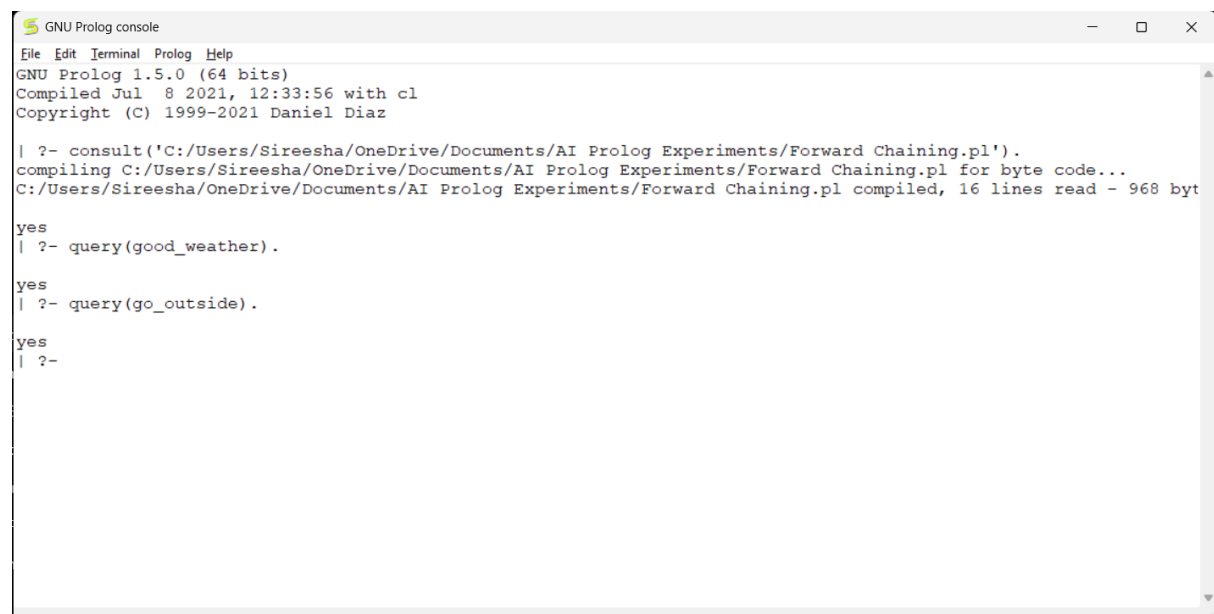
```
% Forward Chaining  
fact(sunny).  
fact(warm).  
rule(good_weather) :-  
    fact(sunny),  
    fact(warm).
```

```
rule(go_outside) :-  
    rule(good_weather).  
  
query(X) :-  
    fact(X).  
  
query(X) :-  
    rule(X).
```

Sample Input

```
?- query(good_weather).  
  
?- query(go_outside).
```

Sample Output



```
GNU Prolog console  
File Edit Terminal Prolog Help  
GNU Prolog 1.5.0 (64 bits)  
Compiled Jul  8 2021, 12:33:56 with cl  
Copyright (C) 1999-2021 Daniel Diaz  
  
| ?- consult('C:/Users/Sireesha/OneDrive/Documents/AI Prolog Experiments/Forward Chaining.pl').  
compiling C:/Users/Sireesha/OneDrive/Documents/AI Prolog Experiments/Forward Chaining.pl for byte code...  
C:/Users/Sireesha/OneDrive/Documents/AI Prolog Experiments/Forward Chaining.pl compiled, 16 lines read - 968 byt  
  
yes  
| ?- query(good_weather).  
  
yes  
| ?- query(go_outside).  
  
yes  
| ?-
```

Result

Thus, the Prolog program for Forward Chaining was successfully implemented and the required conclusions were derived from the given facts and rules.

Ex.No:30

Experiment : Prolog Program for Backward Chaining

Aim

To write a Prolog program to implement Backward Chaining by proving a goal using available facts and rules.

Objectives

- To understand Backward Chaining.
- To define facts and rules.
- To prove a given goal.
- To perform the required queries.

Algorithm

1. Start.
2. Define facts and rules and Select a goal to prove.
3. Check whether the goal is a known fact.
4. If not, find a rule that can prove the goal.
5. Prove the conditions of that rule.
6. If all conditions are true, the goal is true.
7. Display the result.
8. Stop.

Program

```
% Backward Chaining
```

```
bird(parrot).
```

```
has_feathers(parrot).
```

```
can_fly(X) :-
```

```
    bird(X),
```

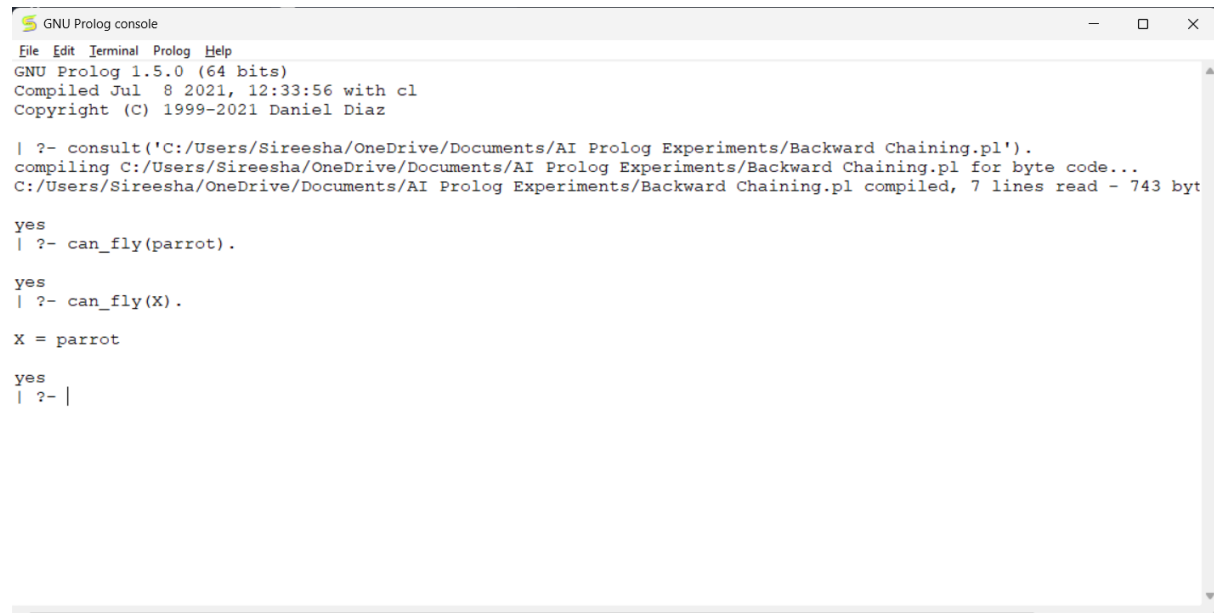
```
    has_feathers(X).
```


Sample Input

?- can_fly(parrot).

?- can_fly(X).

Sample Output



```
GNU Prolog console
File Edit Terminal Prolog Help
GNU Prolog 1.5.0 (64 bits)
Compiled Jul  8 2021, 12:33:56 with cl
Copyright (C) 1999-2021 Daniel Diaz

| ?- consult('C:/Users/Sireesha/OneDrive/Documents/AI Prolog Experiments/Backward Chaining.pl').
compiling C:/Users/Sireesha/OneDrive/Documents/AI Prolog Experiments/Backward Chaining.pl for byte code...
C:/Users/Sireesha/OneDrive/Documents/AI Prolog Experiments/Backward Chaining.pl compiled, 7 lines read - 743 byt

yes
| ?- can_fly(parrot).

yes
| ?- can_fly(X).

X = parrot

yes
| ?- |
```

Result

Thus, the Prolog program for Backward Chaining was successfully implemented, and the given goal was proved using facts and rules.

Ex.No:31

Experiment : Create a Web Blog Using WordPress to Demonstrate Anchor Tag, Title Tag, etc.

Aim

To create a simple web blog using WordPress and demonstrate HTML elements such as the Title Tag, Anchor Tag, Heading Tag, Paragraph Tag, and Image.

Objectives

- To understand the basic features of WordPress.
- To create and publish a simple blog.
- To demonstrate the use of HTML tags.

- To add hyperlinks using the Anchor Tag.
- To use title and heading elements.

Algorithm

1. Start.
2. Open WordPress and create a new website/blog.
3. Create a new blog post.
4. Add a suitable title.
5. Add headings and paragraphs.
6. Add an image to the blog.
7. Add a hyperlink using the Anchor Tag.
8. Preview the blog.
9. Publish the blog.
10. Stop.

Program / HTML Code

The following HTML can be used in a Custom HTML block in WordPress:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
    <title>My Educational Blog</title>
```

```
</head>
```

```
<body>
```

```
    <h1>Welcome to My Educational Blog</h1>
```

```
    <h2>About My Blog</h2>
```

<p>

This blog provides useful information about
computer science and technology.

</p>

<h2>Useful Website</h2>

<p>

Visit

WordPress

to create your own blog.

</p>

<h2>My Topics</h2>

<p>

Python, Artificial Intelligence, Machine Learning
and Web Development.

</p>

</body>

</html>

Sample Input

Blog Title: My Educational Blog

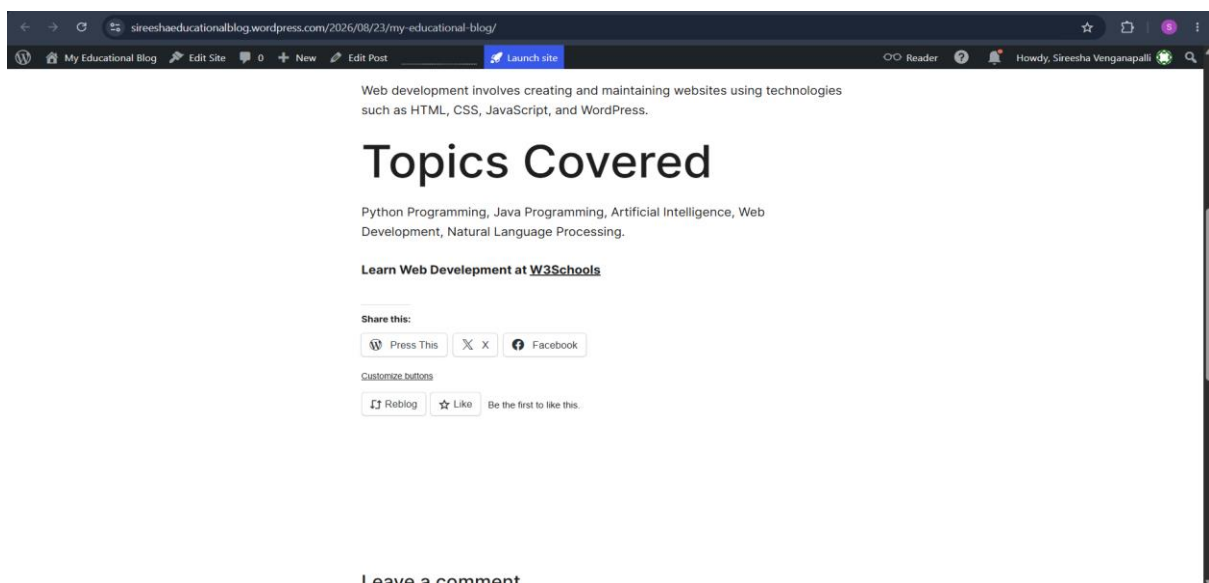
Heading: Welcome to My Educational Blog

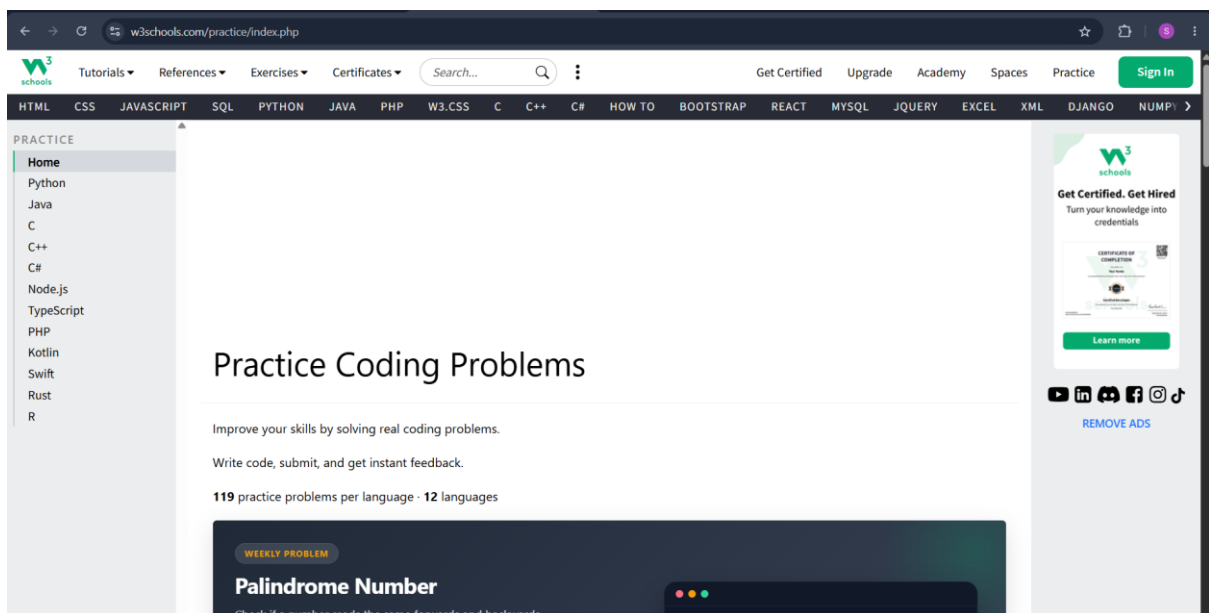
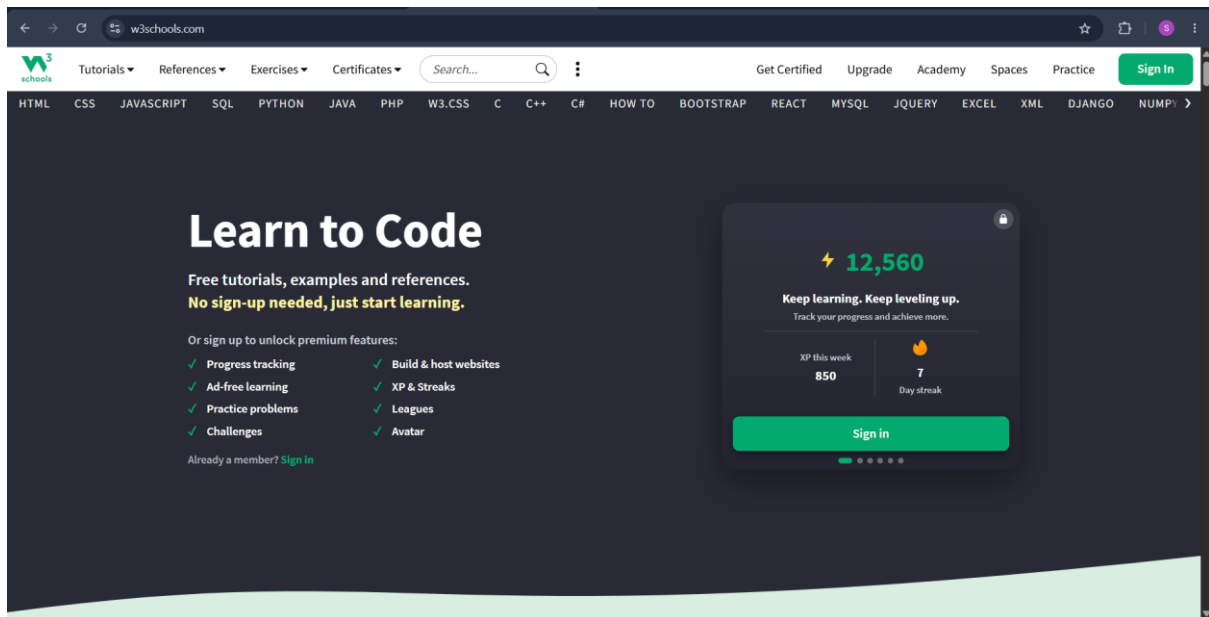
Content:

This blog provides useful information about computer science and technology.

Anchor Text: WordPress

Sample Output





Result

Thus, a simple web blog using WordPress was successfully created, demonstrating the Title Tag, Heading Tags, Paragraph Tag, Anchor Tag, and other basic HTML elements.